
Kinetic-KAN: Solver-Aware Neural Dynamics

Project Final Report

Kolmogorov-Arnold Network Neural ODEs — cross-domain stability fixes, solver-order effects, basis-function design, and a stiffness-solver stability phase map, verified against real training runs, checkpoints and figures.

Submitted By

- 2105032** Nawriz Ahmed Turjo — Adjoint vs. Autograd Profiling
2105033 Abhishek Roy — Gradient Norm Dynamics
2105048 Shams Hossain Simanto — Learnable Basis Blending
2105055 Md. Abrar Jahin — Stiffness–Solver Stability Analysis
2105043 Monjur Hossain Khan (Shovon) — Sparse Regression & Epidemic Fit

September 21, 2026

Executive Summary

This report documents a Kolmogorov-Arnold Network Neural ODE (KAN-ODE) [4, 6, 2] project in four stages: a baseline recipe reproducing the base paper’s Lotka-Volterra and SIR benchmarks; a diagnosis-and-fix cycle that resolved two cross-domain training failures (damped pendulum, SIR); five independent follow-on studies probing solver order, differentiation method, basis-function design, and stiffness; and a validation stage (epoch-budget sensitivity, paper-implementation audit) whose infrastructure is complete but whose runs have not yet executed.

- The pendulum failure ($R^2 = -1.318$) was fixed by extending the training window, reaching $R^2 = +0.647$ — a $20\times$ full-horizon MSE improvement. SIR’s failure traced to a unit-scale mismatch between the network’s initial field and the true dynamics.
- Adjoint sensitivity is strictly worse than direct autograd at this project’s scale: it already uses less memory (17.1 vs. 18.6–21.5 MB) but costs $1.9\text{--}2.1\times$ wall-clock, and no crossover point exists inside four orders of magnitude of trajectory length.
- Gradient-norm roughness fails to explain Euler’s convergence floor (Spearman $\rho = -0.088$, $p = 0.87$); a shared, order-independent spike-burst structure dominates all six solvers’ gradient traces.
- A learnable spline/RBF basis blend never beats the better pure basis on either system tested, and on the pendulum it overfits catastrophically past $\sim 3,000$ epochs in a way pure RBF does not.
- The stiffness–solver stability analysis traced failures at $\mu \in \{0.5, 1.0, 2.0\}$ to random seed rather than solver order or numerical stiffness; the $\mu = 2.0$ trap is a genuine, solver-independent spurious equilibrium in the learned vector field, confirmed directly from trained weights and from an independent weight-space loss-landscape analysis.
- SINDy is far more noise-robust than KAN-ODE ($9\times$ vs. $1540\times$ degradation, $\sigma = 0 \rightarrow 0.1$) but strictly less accurate at every noise level tested; a real, non-mechanistic epidemic curve confirmed that two of SIR’s three stability fixes transfer by the same mechanism that fixed SIR itself.

Every number and figure in this report is traceable to a saved training record or a regenerable figure from this project’s own experiment logs.

Contents

Executive Summary	1
1 Introduction	3
1.1 Background: Kolmogorov-Arnold Networks as vector fields	3
1.2 What the base paper fixed, and what this project varies	3
1.3 Why training itself needs to be treated as an object of study	3
1.4 Structure of this report	4
2 Baseline Recipe and Evaluation Protocol	4
3 Benchmark Ablation and Cross-Domain Stability Analysis	4
3.1 Solver, basis and architecture ablation	5
3.2 Diagnosing and fixing two cross-domain failures	6
4 Adjoint Sensitivity vs. Direct Autograd: Memory and Wall-Clock Cost	8
5 Gradient-Norm Dynamics and Solver Order	9
6 Learnable Basis-Function Blending	10
7 Stiffness–Solver Stability and Loss-Landscape Analysis	12
7.1 The stability question resolves quickly	12
7.2 What actually drives the failures: initialization	13
7.3 The mechanism: a genuine spurious equilibrium	14
7.4 Weight-space loss landscape: an independent measurement	14
8 Sparse Regression Comparison and Real-World Epidemic Validation	18
8.1 Noise robustness: KAN-ODE vs. sparse symbolic regression	18
8.2 Edge pruning: testing whether a term-by-term comparison is even possible	19
8.3 Testing whether SIR’s fixes transfer to a non-mechanistic epidemic curve	19
9 Validation Status: Epoch-Budget Sensitivity and Implementation Audit	21
9.1 Epoch-budget sensitivity check	21
9.2 Baseline-vs-paper implementation audit	21
10 Discussion	21
11 Conclusion	22

1 Introduction

1.1 Background: Kolmogorov-Arnold Networks as vector fields

A standard Neural ODE [2] approximates an unknown dynamical system $\dot{\mathbf{u}}(t) = \mathbf{F}(\mathbf{u}(t))$ by a learned vector field $\mathbf{f}_\theta(\mathbf{u}(t))$, parameterized by a multilayer perceptron: fixed nonlinear activations on each node, with all the learnable structure sitting in the linear weights between them, $\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$. A Kolmogorov-Arnold Network (KAN) [6] inverts this: every *edge* carries its own learnable univariate function $\phi(x)$, and each node simply sums its incoming edges, $y_j = \sum_i \phi_{i,j}(x_i)$. This construction is motivated by the Kolmogorov-Arnold representation theorem, which states that any continuous multivariate function on a bounded domain can be written as a finite composition of continuous univariate functions and addition,

$$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right), \quad (1)$$

so that, in principle, no fixed nonlinearity needs to be chosen in advance – the network can learn what shape each univariate function should take.

Each edge function in a practical KAN is not learned freely; it is represented as a fixed smooth residual plus a learnable expansion in a chosen basis,

$$\phi(x) = w_b b(x) + w_s \sum_{i=1}^{G+k} c_i B_i(x), \quad b(x) = \text{silu}(x) = \frac{x}{1 + e^{-x}}, \quad (2)$$

where $B_i(x)$ are the $G + k$ basis functions spanning a grid of G intervals, c_i are trainable coefficients, and w_b, w_s trade off the fixed residual against the learned expansion. The choice of B_i is a design decision, not a mathematical necessity: a Gaussian radial basis function (RBF) grid, $B_i(x) = \exp(-(x - \mu_i)^2 / 2\sigma^2)$, gives each edge smooth global support, while a degree- k B-spline built by the Cox-de Boor recursion gives each edge compact local support, zero outside its own knot span. KAN-ODEs [4] put this construction inside a Neural ODE’s vector field: $\dot{\mathbf{u}} = \mathbf{f}_\theta(\mathbf{u})$ with every edge of $\mathbf{f}_\theta$ built from Equation 2, and the resulting trajectory obtained by numerically integrating that field forward in time.

1.2 What the base paper fixed, and what this project varies

The base paper reports two central claims on the Lotka-Volterra predator-prey system: a KAN-ODE with 240 parameters extrapolates Lotka-Volterra’s limit cycle accurately from a short training window, and it reaches a target training loss in roughly 10^4 epochs against a parameter-matched multilayer-perceptron Neural ODE’s 10^5 – a claimed tenfold training-speed advantage at 252 parameters. To obtain these results, the paper commits to three specific design decisions that this project treats as free variables rather than fixed choices: the **integrator** (an adaptive fifth-order Runge-Kutta method, Tsit5), the **basis** (Gaussian RBF, chosen in part because a smooth global basis supports the paper’s longer-term goal of recovering closed-form symbolic laws from the trained edges), and the **differentiation method** used to compute gradients through the integrator (continuous adjoint sensitivity, motivated by its constant memory cost regardless of trajectory length).

Each of those three fixed choices maps directly onto one of this report’s studies: Section 4 tests whether the adjoint method’s memory motivation actually holds at this project’s scale; Sections 3, 5, and 7 test the integrator choice, sweeping solver order and, in the stiffness study, solver order jointly with a physical stiffness parameter; and Section 6 tests the basis choice directly, by making the basis itself a learnable, trained quantity rather than a fixed decision made in advance. Section 8 extends the comparison outside the base paper’s own scope entirely, benchmarking KAN-ODE against a sparse-regression alternative built for the same symbolic-discovery goal the RBF basis was partly chosen to support, on both synthetic data and a real, non-mechanistic epidemic curve.

1.3 Why training itself needs to be treated as an object of study

This project uses three testbeds beyond the base paper’s own Lotka-Volterra system: the SIR epidemic model and a damped pendulum. Both were chosen because their governing equations are simple and well understood, which makes any training failure attributable to the architecture or the optimization process rather than to an ambiguous or poorly specified target dynamics. The integrator’s interaction with training turns out not to be

predictable from classical numerical-analysis theory alone, because the integrator here sits inside a backpropagation graph rather than a standalone forward solve: a solver that is unconditionally stable in the classical sense can still produce a network that trains badly, trains to a wrong answer, or trains well but fails to generalize. Diagnosing failures of this kind requires treating the training process itself as an object of study, not just the final trajectory error.

1.4 Structure of this report

The report proceeds in the order the work was actually carried out. A baseline recipe is reproduced from the original KAN-ODE paper and used to establish this project’s evaluation protocol (Section 2). That recipe is then stress-tested with a full solver and basis-function ablation, which surfaces two outright training failures on the new systems; both are diagnosed to a specific mechanistic cause and fixed (Section 3). Five further studies build on that fixed baseline, each isolating one of the questions raised above: computational cost of the gradient method (Section 4), whether gradient noise explains a solver-order effect (Section 5), whether a learnable blend of basis functions helps (Section 6), how solver choice and stiffness interact with training stability (Section 7), and how KAN-ODE compares against a classical sparse-regression method on synthetic and real data (Section 8). A validation stage (Section 9) checks two remaining assumptions behind the benchmark ablation. Section 10 draws together the patterns that recur across these otherwise-independent studies, and Section 11 closes the report.

2 Baseline Recipe and Evaluation Protocol

The baseline stage reproduces the base paper’s KAN-ODE recipe [4] on the Lotka-Volterra predator-prey system,

$$\dot{u}_1 = \alpha u_1 - \beta u_1 u_2, \quad \dot{u}_2 = \delta u_1 u_2 - \gamma u_2, \quad (\alpha, \beta, \gamma, \delta) = (1.5, 1.0, 3.0, 1.0), \quad (3)$$

using a two-layer Kolmogorov-Arnold Network, $[2, 10, 2]$, with $G = 5$ Gaussian radial-basis-function grid points per edge (240 parameters), trained on $t \in [0, 3.5]$ (36 points) and scored strictly on $t \in (3.5, 14.0]$ (105 unseen points, approximately four limit-cycle periods). Extrapolation is scored this way, on points the training loss never touched, because it is the only meaningful test of whether the network learned the underlying dynamics rather than memorized the shape of the training window; a model that only reproduces the 36 training points could still be badly wrong about the system it is meant to describe.

A parallel Susceptible-Infected-Recovered (SIR) compartmental epidemic model and a damped-pendulum system were carried forward as the project’s two additional test systems, chosen for a specific reason beyond simply adding more benchmarks: both have closed-form governing equations that are textbook-simple and well understood, so that if training fails on either one, the failure can be attributed to the architecture or the optimization process itself rather than to an ambiguous, hard-to-verify target dynamics. SIR additionally has a real algebraic invariant (its three compartments sum to a constant), and the pendulum has a real, correctly-known asymptotic equilibrium – both properties later sections lean on directly (Sections 3 and 7).

This stage established the project’s evaluation protocol and the two metrics used throughout this report. Model selection uses minimum *training*-window loss only, never by inspecting performance on the extrapolation window; the reported error is the mean squared error (MSE) between the model’s predicted trajectory and the true trajectory, computed strictly outside the training horizon, so that a number cannot improve simply by widening what counts as fit. The coefficient of determination, R^2 , reports the same comparison on a scale where 1 is a perfect match and 0 is no better than predicting the mean of the true trajectory; a negative value means the model performs worse than that trivial baseline. Every subsequent study in this report inherits this protocol unchanged.

3 Benchmark Ablation and Cross-Domain Stability Analysis

Before extending KAN-ODE to new systems, two questions needed settling: which solver and basis choices this architecture actually prefers, and whether the baseline recipe (Section 2) transfers unchanged to systems beyond Lotka-Volterra. Both were answered empirically, and the second uncovered two genuine training failures that required a mechanistic diagnosis before they could be fixed.

3.1 Solver, basis and architecture ablation

Table 1 sweeps six explicit Runge-Kutta integrators at fixed radial-basis-function (RBF) basis; Table 2 sweeps seven basis representations at fixed Tsit5 solver. Both use the protocol of Section 2. The number of function evaluations per training epoch (how many times the integrator must call the network per step) is reported because it is the dominant cost driver for training time. The Lipschitz constant L of the trained field – an upper bound on how much the output of $\mathbf{f}_\theta$ can change per unit change of its input, estimated numerically from the trained weights – is reported alongside accuracy because it is a direct measure of how smooth or jagged the basis expansion in Equation 2 ended up, independent of the trajectory error.

Table 1. Solver ablation (fixed RBF basis, $\Delta t = 0.1$, 240 parameters).

Solver	Order	Function evaluations / epoch	Training MSE	Extrapolation MSE	Extrapolation R^2	Lipschitz L
Forward Euler	1	70	1.38×10^{-4}	2.51×10^{-3}	0.99914	8.55
Heun RK2	2	140	8.32×10^{-5}	3.10×10^{-4}	0.99989	7.07
Midpoint	2	140	8.86×10^{-5}	8.95×10^{-5}	0.99997	6.96
Classical RK4	4	280	9.41×10^{-5}	9.29×10^{-5}	0.99997	6.88
DOPRI5	5	420	9.28×10^{-5}	9.21×10^{-5}	0.99997	6.89
Tsit5	5	420	8.83×10^{-5}	8.92×10^{-5}	0.99997	6.91

Truncation error dominates only at order $p = 1$: Euler’s extrapolation MSE is $28\times$ Tsit5’s, and it is the only solver that fails to reach 10^{-4} training loss within budget. Its trained field is also the least smooth of the six ($L = 8.55$, the highest in the table), consistent with a systematic bias forcing the network to fit a sharper correction than the smoother integrators require. Above order 2 the ranking collapses into a 3.8% spread with no monotonic order dependence – at $\Delta t = 0.1$, integrator truncation error has fallen below the optimization noise floor. Midpoint matches Tsit5’s accuracy within 0.3% at one third of the function evaluations, making it the cost-normalized optimum on this problem.

Table 2. Basis ablation (fixed Tsit5 solver, $G = 5$ grid points, 240 parameters).

Basis function	Training MSE	Extrapolation MSE	R^2	Lipschitz L	Wall-clock time
Cubic B-spline	7.20×10^{-5}	8.38×10^{-5}	0.99997	8.44	6680 seconds
Gaussian RBF	8.83×10^{-5}	8.92×10^{-5}	0.99997	6.91	1712 seconds
Chebyshev	9.80×10^{-5}	4.37×10^{-4}	0.99985	6.48	2317 seconds
Lagrange	1.07×10^{-4}	3.93×10^{-4}	0.99987	6.70	6829 seconds
Newton	1.47×10^{-3}	3.65×10^{-2}	0.98746	5.65	2187 seconds

B-spline edges out RBF on accuracy after a boundary-knot fix, at $3.9\times$ RBF’s wall-clock time and a visibly higher Lipschitz constant (8.44 vs. 6.91) – its compact local support (Equation 2’s $B_i(x)$ vanishing outside each knot span) lets it fit sharper local features than RBF’s smooth global support without being penalized elsewhere in the domain. The two bases are statistically tied on error (6% apart, inside single-seed noise), which makes RBF the better accuracy-per-second choice. Polynomial global-support bases (Chebyshev, Lagrange) fit the training window well and extrapolate $4\text{--}5\times$ worse – the expected signature of a basis whose every coefficient influences the whole domain rather than a local region of it. Newton’s basis is the outlier on smoothness in the opposite direction: its lowest Lipschitz constant in the table (5.65) reflects underfitting, not desirable flatness, since it is also the only basis that never reaches 10^{-3} training loss.

A parameter-matched comparison against a standard multilayer perceptron (MLP) trained as a Neural ODE found KAN-ODE extrapolates $5.4\times$ better than the MLP from statistically identical training loss, while the MLP converges $4.1\times$ faster to 10^{-3} loss. This directly contradicts the base paper’s own headline efficiency claim on this same system – a 10^4 -epoch KAN-ODE against a 10^5 -epoch, parameter-matched MLP-ODE, a tenfold speed advantage the paper attributes to the network (Section 1). Under this project’s protocol, at matched parameter count (240 vs. 252) and matched optimizer settings, the MLP was faster to converge on Lotka-Volterra rather than slower, though KAN-ODE still won decisively on the extrapolation accuracy the paper’s other headline claim concerns.

Table 3. Controlled breakdown of why the paper’s literal MLP baseline architecture, $[2, 50, 2]$ with hyperbolic tangent (tanh) activation, fails to train under this project’s protocol. All four runs use 2000 epochs.

Run	Architecture	Activation	Learning rate	Training MSE	Extrapolation MSE	R^2
A	$[2, 50, 2]$	tanh	2×10^{-3}	1.21×10^0	1.98×10^0	0.319
B	$[2, 50, 2]$	tanh	5×10^{-4}	1.85×10^0	9.23×10^0	-2.169
C	$[2, 14, 8, 8, 2]$	SiLU	2×10^{-3}	4.28×10^{-4}	6.97×10^{-4}	0.99976
D	$[2, 50, 2]$	SiLU	2×10^{-3}	3.62×10^{-3}	2.68×10^2	-91.06

Comparing run A against run D, which share the same architecture and differ only in activation, shows a $335\times$ gap in extrapolation error – the activation function determines whether this architecture fits the data at all. Comparing run C against run D, which share the SiLU activation and differ only in architecture depth, shows extrapolation error of 6.97×10^{-4} against 2.68×10^2 – network depth determines extrapolation quality once the activation is fixed to one that trains. This isolates the activation choice as the specific cause of the paper’s literal architecture failing to train under this project’s protocol, independent of the epoch budget question addressed in Section 9.

3.2 Diagnosing and fixing two cross-domain failures

Applying the Lotka-Volterra recipe unchanged to the damped pendulum and to SIR failed on both systems, each for its own reason. Both diagnoses follow the same discipline: measure a quantity about the model’s behavior *before* training rather than search the hyperparameter space after a failed run.

Pendulum: extending the training window. The pendulum recipe originally trained on $t \in [0, 3.5]$ and diverged in extrapolation ($R^2 = -1.318$). Extending the training window to $t_{\text{train}} = 5.0$ – a single flag – resolves it:

Question	Why does the Lotka-Volterra recipe, applied unchanged, diverge on the pendulum?
Expected (hypothesis)	SiLU’s residual branch cannot represent the sign-changing derivative $\dot{\theta} = \omega$ – an architectural limit.
Observed	Refuted. A longer training window fixes it under either activation; SiLU fits the sign change fine once the window is long enough.

Table 4. Pendulum fix: best configuration versus the original failure.

Configuration	Full-horizon MSE	Extrapolation R^2	Verdict
Original ($t_{\text{train}} = 3.0$)	8.99×10^{-1}	-1.318	diverges
Extended window ($t_{\text{train}} = 5.0$, SiLU)	4.48×10^{-2}	+0.647	adopted

An earlier hypothesis held that SiLU’s residual branch $W \cdot \text{silu}(x)$ cannot represent a sign-changing derivative such as $\dot{\theta} = \omega$. A factorial control comparing activation against training-window length refutes it directly: with the longer window, SiLU reaches an angular-velocity root-mean-square error of 0.0114 radians and reproduces the true turning point ($\theta_{\text{min}} = -1.380$ against a true value of -1.385). The training window is what fixes the pendulum; the activation choice affects only how quickly the fit is reached, with the identity activation converging $\sim 1000\times$ faster at the short window without matching the longer window’s accuracy.

SIR: a unit-scale mismatch. SIR diverged to an invalid (not-a-number) value at epoch 8686.

Question	What causes SIR’s divergence, and is it a scale problem, an optimizer defect, or a modeling gap?
Expected	Measuring the untrained network’s own field magnitude against the true dynamics, before any training, should reveal whether the two are mismatched in scale.
Observed	A $19\times$ speed mismatch at initialization. Rescaling time removes the divergence entirely; no optimizer or architecture change was needed.

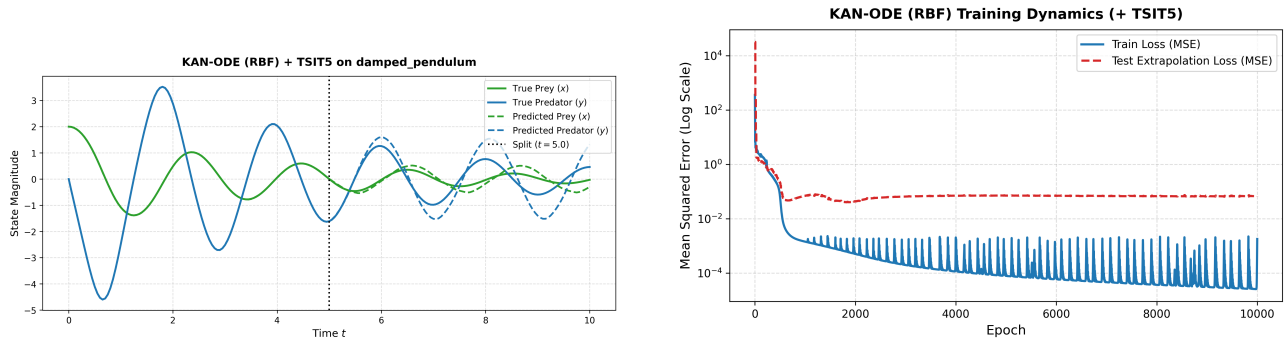


Figure 1. Fixed-pendulum trajectory reconstruction (left) and training/extrapolation loss curves (right), using the extended-window recipe.

Measuring the learned field magnitude *before any training step* explained it directly: a Glorot-initialised KAN starts with $|f_\theta| \approx 0.203$ against SIR’s true field magnitude of 0.011 over a training horizon $T = 50$ – a $19\times$ speed mismatch that makes $f_\theta \rightarrow 0$ the steepest local descent direction available. The optimizer was behaving rationally: collapsing the field to zero genuinely reduces the loss fastest when the target dynamics evolve this much slower than the model’s initial guess. Rescaling time by a factor of 10 removes the mismatch; projecting the learned field onto the subspace where the compartments sum to a constant fixes a separate mass-drift symptom. Final SIR result: mass error reduced from 5.9×10^{-2} to 7×10^{-7} , extrapolation $R^2 = +0.9981$.

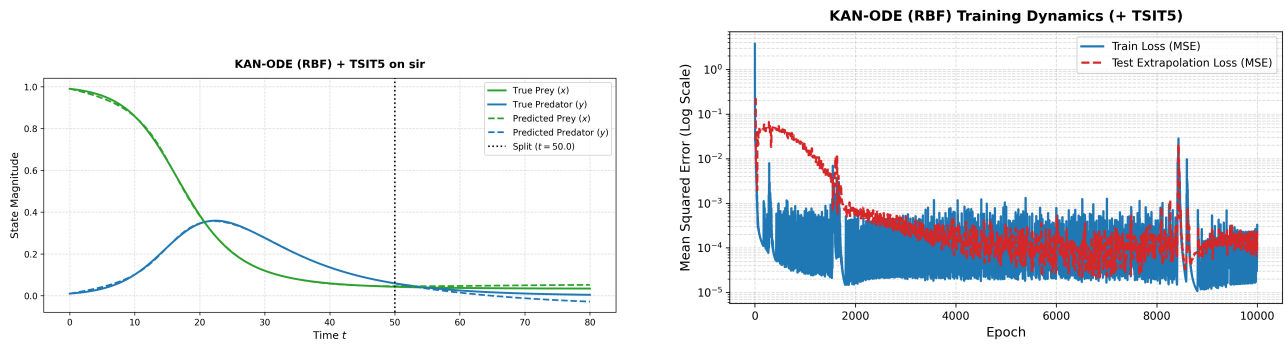


Figure 2. Fixed-SIR trajectory reconstruction (left) and loss curves (right).

Other fixes considered. Several additional interventions were tested and are recorded for completeness rather than adopted:

Table 5. Every stability intervention tested, including the rejected ones.

Intervention	Verdict	Evidence
Longer training window (pendulum)	Adopted	R^2 improves from -1.318 to $+0.647$; works for both activations tested
Non-finite gradient guard with automatic abort	Adopted	Caught SIR’s divergence at epoch 3055, preserved a valid checkpoint instead of overwriting it with an invalid one
Time rescaling (SIR)	Adopted	Removes both the divergence and the frozen-field behavior
Conservation via subspace projection (SIR)	Adopted	Mass error reduced from 5.9×10^{-2} to 7×10^{-7}
Identity activation (pendulum)	Speed only	Roughly $1000\times$ faster to fit at the short window, but worse extrapolation and stability at the long window
Standard-deviation loss weighting	System-dependent	Worst-performing pendulum option; superseded for SIR by time rescaling
Conservation via penalty term	Rejected	Wrong mechanism – the training window’s mass was already close to conserved, so the penalty had nothing to correct
Widened basis-function grid limits	Rejected	Performed worse than the untouched baseline configuration

Both diagnoses came from measuring the learned field’s own magnitude at initialization, before training ever ran. This init-time-measurement discipline reappears verbatim in the real-epidemic-data test of Section 8.

4 Adjoint Sensitivity vs. Direct Autograd: Memory and Wall-Clock Cost

Two methods exist for computing gradients through a neural ODE. **Direct autograd** treats the numerical integrator as an ordinary computational graph: each of the solver’s N_t forward steps is recorded, and the reverse pass walks back through every recorded step in turn, so peak memory grows linearly with the number of steps taken. **Adjoint sensitivity** [2] instead treats the forward solve as a black box and derives a second, augmented ODE that produces the same gradient without storing every intermediate state. Writing $\mathbf{a}(t) = \partial L / \partial \mathbf{u}(t)$ for the adjoint state, it satisfies its own backward-time differential equation,

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^\top \frac{\partial \mathbf{f}_\theta}{\partial \mathbf{u}}(\mathbf{u}(t)), \quad \frac{dL}{d\theta} = \int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial \mathbf{f}_\theta}{\partial \theta}(\mathbf{u}(t)) dt, \quad (4)$$

solved backward from t_1 to t_0 alongside the original state $\mathbf{u}(t)$ (recomputed rather than stored), so memory depends only on the size of the state and parameter vectors, not on how many steps the forward solve took – an $O(1)$ memory cost in trajectory length, against direct autograd’s $O(N_t)$. This project’s own implementation documentation adopts direct autograd on the stated grounds that this $O(1)$ advantage does not matter for this project’s short trajectories ($N_t \in [36, 101]$). This section measures that claim directly on an RTX 3060, holding the integrator fixed (classical RK4 at matched step size on both paths; a forward-trajectory check confirms the two implementations agree to $\sim 10^{-7}$ relative error) so that only the differentiation method varies.

Question	Does adjoint sensitivity’s constant memory cost make it the better choice at this project’s trajectory lengths, as the project’s own documentation assumes?
Expected	Direct autograd should be cheaper in memory at these short lengths, with adjoint’s advantage only appearing once trajectories grow much longer.
Observed	Adjoint already uses less memory at every length tested. The deciding cost is wall-clock, not memory: adjoint is $1.9\text{--}2.1\times$ slower.

Table 6. Peak VRAM and wall-clock, direct autograd vs. adjoint sensitivity.

System	Trajectory length N_t	Method	Peak VRAM (MB)	Seconds per 1000 epochs
Lotka-Volterra	36	Direct autograd	18.61	317.5
Lotka-Volterra	36	Adjoint	17.08	672.9
SIR	101	Direct autograd	21.52	984.8
SIR	101	Adjoint	17.12	1871.6

Finding. The measured memory data contradicts the stated premise behind the original choice, while still supporting its conclusion. The $O(N_t)$ vs. $O(1)$ scaling in Equation 4 predicts a crossover length below which direct autograd’s smaller constant factor should still win on raw memory, since it avoids the adjoint method’s own bookkeeping overhead; the data shows no such crossover inside the tested range. Adjoint uses *less* memory than direct autograd at every tested length, including the shortest ($N_t = 36$: 17.08 vs. 18.61 MB) – Figure 3 shows direct autograd’s per-step storage cost is already large enough at this project’s scale to outweigh whatever constant overhead the adjoint method’s own bookkeeping carries. Extrapolating the measured linear slope of direct autograd’s memory growth, N_t would need to reach $\approx 225,000$ before it threatens a 12 GB card. The deciding factor for this project is wall-clock: solving the continuous adjoint ODE backward in time costs a consistent $1.9\text{--}2.1\times$ overhead at both tested lengths, turning a 53-minute Lotka-Volterra run into ~ 112 minutes for a memory saving that never becomes load-bearing (peak usage is 0.17% of card capacity). Gradient relative

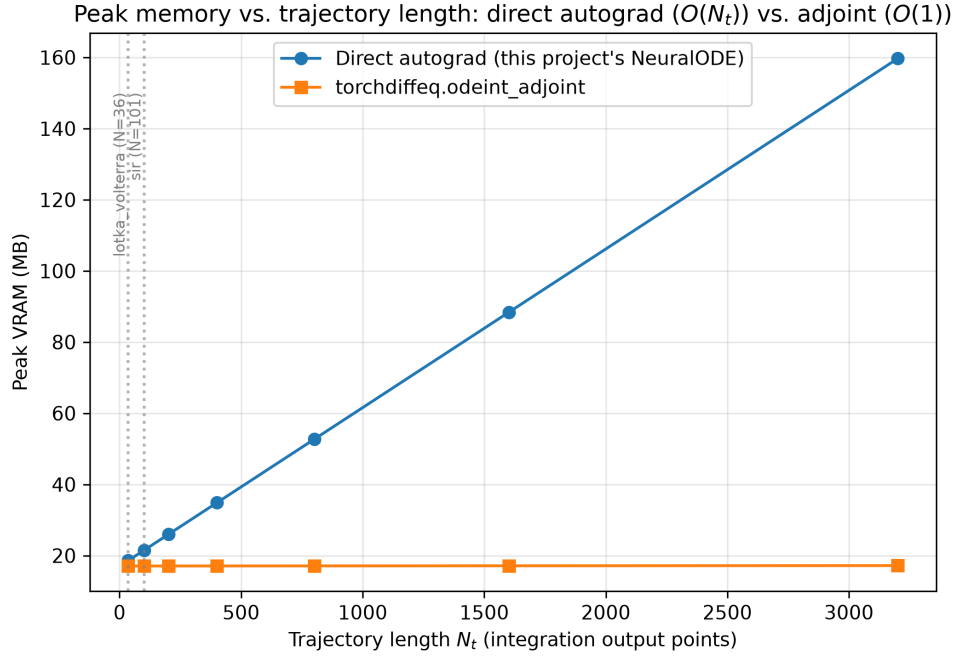


Figure 3. Peak memory vs. trajectory length $N_t \in \{36, \dots, 3201\}$: direct autograd grows linearly (≈ 0.045 MB per output point); adjoint stays flat (0.7% increase over an $89\times$ length increase).

error is 2.85×10^{-7} and 4.88×10^{-7} respectively, well inside a 5×10^{-3} tolerance – both methods agree numerically, so the choice reduces to a resource trade-off. Direct autograd is the faster option for every dataset in this project’s scope ($N_t \leq 201$).

5 Gradient-Norm Dynamics and Solver Order

The benchmark ablation found Forward Euler uniquely unable to reach 10^{-4} training loss within budget, with $28\times$ worse extrapolation MSE than Tsit5 (Table 1). The candidate mechanism tested here: a first-order solver commits $O(\Delta t)$ local error every step, which could inject measurably noisier gradients and destabilize Adam’s moment estimates.

Question	Does a lower-order solver’s larger truncation error show up as measurably noisier gradients during training?
Expected	Gradient-norm noise should fall as solver order rises, correlating with the accuracy ranking already seen in the benchmark ablation.
Observed	No reliable correlation ($\rho = -0.088$, $p = 0.87$); the ranking is non-monotone and its sign flips depending on an arbitrary warm-up cut. Refuted.

The noise measure is the standard deviation of epoch-to-epoch *relative* gradient change, scale-invariant by construction,

$$\eta = \text{std}(\Delta \log_{10} g), \quad \Delta \log_{10} g_i = \log_{10} g_{i+1} - \log_{10} g_i, \quad (5)$$

compared against solver order p by Spearman rank correlation rather than the more familiar Pearson correlation. Pearson correlation measures how linearly two quantities move together, which presumes solver order is a quantity whose actual numeric spacing is meaningful – that the jump from order 1 to order 2 is comparable in size to the jump from order 4 to order 5. There is no reason to expect η to depend on solver order that linearly; the hypothesis under test is only that noise falls *monotonically* as order rises, which is exactly what Spearman correlation measures, by comparing ranks rather than raw values. It also handles the deliberate tie at $p = 2$ (Heun and Midpoint are both second-order methods) natively, where Pearson would need an arbitrary tie-breaking rule.

Spearman $\rho = -0.088$, $p = 0.87$ ($n = 6$). Three findings jointly refute the hypothesis: the effect size is a 7% spread across all six solvers; the ranking is non-monotone (RK4, order 4, is the second-noisiest solver); and the sign of ρ flips across warm-up cuts ($-0.088 \rightarrow -0.441 \rightarrow +0.706$ as the first 0, 2000, 5000 epochs are

Table 7. Gradient-noise measure η vs. solver order, six solvers, $n = 6$.

Solver	Order p	Noise η	Best train MSE	Noise rank
Forward Euler	1	0.2805	1.377×10^{-4}	1 (noisiest)
Classical RK4	4	0.2726	9.413×10^{-5}	2
Tsit5	5	0.2683	8.833×10^{-5}	3
DOPRI5	5	0.2660	9.283×10^{-5}	4
Midpoint RK2	2	0.2634	8.857×10^{-5}	5
Heun RK2	2	0.2621	8.315×10^{-5}	6

dropped), so no cut reaches significance and the direction of any relationship depends on an arbitrary analyst choice.

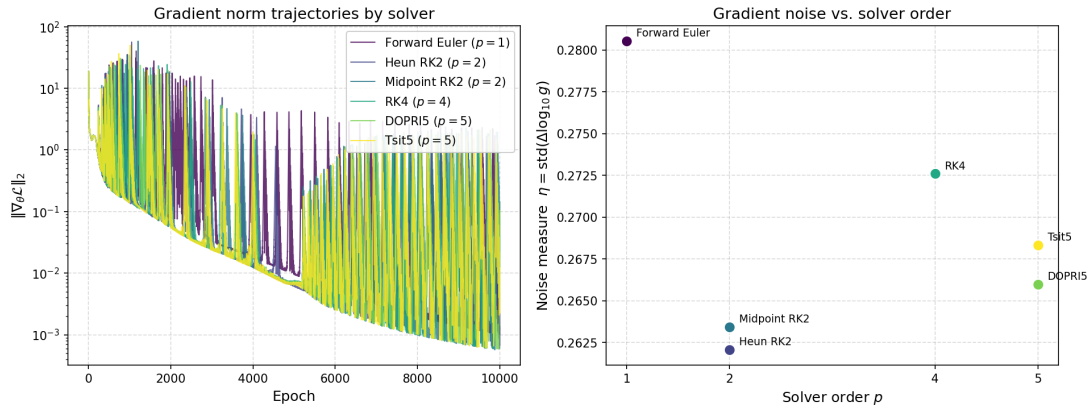


Figure 4. Gradient-norm traces, all six solvers: large sustained spike bursts (one to two orders of magnitude above the local median) dominate every trace, including Tsit5, independent of solver order.

Finding. Gradient-norm roughness does not explain Euler’s convergence floor. All six traces are instead dominated by shared, order-independent spike bursts (1176 spike-epochs for Euler, 866 for Tsit5 in the last 4000 epochs, near-identical burst structure). Four checks rule out a numerical defect: zero non-finite gradients or losses across 60,000 logged epochs; loss decreases monotonically in its running minimum for every solver; loss is lower five epochs after a spike in 64.5% of cases; and median loss is elevated on spike epochs (3.90×10^{-4} vs. 1.80×10^{-4} on calm epochs), ruling out a spurious-gradient/still-loss signature. The most plausible reading is edge-of-stability optimizer behaviour: gradient descent with a fixed step size and no learning-rate schedule is known to settle into a regime where it repeatedly overshoots the walls of a narrowing loss valley as training approaches a sharp minimum, producing exactly this pattern of a large transient spike followed by fast recovery, rather than either smooth convergence or a genuine divergence. Because this behavior appears with near-identical structure in Euler’s and Tsit5’s traces alike, it is a property of the optimizer and the loss surface’s local geometry, not of the integrator’s truncation error. Euler’s convergence floor is better explained by the earlier systematic-bias hypothesis (Section 3): a first-order integrator’s $O(\Delta t)$ local error biases *where* the optimizer settles, without making the path it takes to get there measurably rougher.

6 Learnable Basis-Function Blending

The benchmark ablation found B-spline (compact local support, $3.9\times$ RBF’s cost) and RBF (smooth global support, cheap) statistically tied on accuracy. Rather than choosing one basis function $B_i(x)$ in Equation 2 in advance, this study makes the choice itself a trained quantity: a learnable convex blend of the two candidate bases inside every edge,

$$\phi(x) = \alpha \cdot \text{Spline}(x) + \beta \cdot \text{RBF}(x), \quad (\alpha, \beta) = \text{softmax}(\text{logits}), \quad (6)$$

trained end-to-end from a neutral start ($\alpha = \beta = 0.5$) with no hand-set schedule, on Lotka-Volterra and the stability-fixed pendulum (Section 3). The blend weights are produced by a softmax over two learnable logits rather than, say, a single sigmoid-gated weight, because a softmax guarantees $\alpha + \beta = 1$ by construction – the two bases always form a genuine convex combination, so the gate can only trade one basis off against the other rather than scaling the overall edge output up or down as a side effect. A single pair of logits is shared across every edge in the network, so the whole model learns one global blend rather than letting each edge pick its own; this was a deliberate simplification to keep the number of new parameters at 2, not a claim that per-edge blending would not also work. If a single global blend systematically beats both pure bases, it would suggest the basis-function choice in Equation 2 is itself worth learning rather than fixing by hand, for every future KAN-ODE application.

Question	Can a network do better by learning its own blend of B-spline and RBF, rather than committing to one basis in advance?
Expected	A learned blend should match or beat the better pure basis, by adaptively favoring local support where useful and global support elsewhere.
Observed	Hybrid never beats the better pure basis on either system. On the pendulum, it actively overfits past 3000 epochs in a way pure RBF does not.

Table 8. Hybrid basis vs. pure bases, full 10,000-epoch budget.

System / basis	Training MSE	Extrapolation MSE	R^2	Final gate
Lotka-Volterra – RBF (pure)	8.83×10^{-5}	8.92×10^{-5}	1.0000	–
Lotka-Volterra – B-spline (pure)	7.20×10^{-5}	8.38×10^{-5}	1.0000	–
Lotka-Volterra – Hybrid	1.47×10^{-4}	1.78×10^{-4}	0.9999	$\alpha=0.12, \beta=0.88$
Pendulum – RBF (pure)	9.28×10^{-5}	0.090	+0.647	–
Pendulum – Hybrid (10,000 epochs)	9.44×10^{-5}	0.809	−2.17	$\alpha=0.02, \beta=0.98$

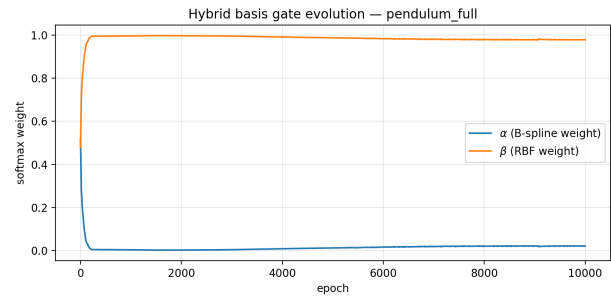
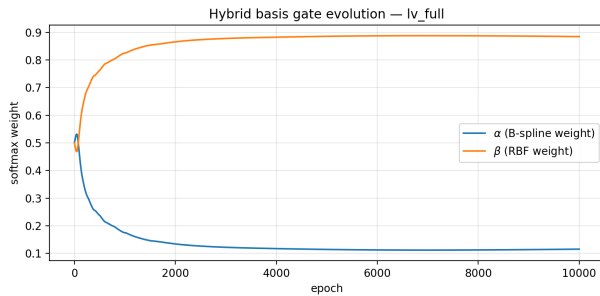


Figure 5. Learned α/β gate evolution: Lotka-Volterra settles into a genuine partial blend (left); the pendulum gate converges to $\approx 98\%$ RBF by epoch 250 (right), well before the overfitting onset discussed below.

Lotka-Volterra: no benefit, real cost. The gate settles into a genuine partial blend, retaining $\approx 12\%$ B-spline, which confirms the mechanism works as designed. Hybrid ties both pure bases on final accuracy while paying roughly the *sum* of their per-epoch compute.

Pendulum: overfitting emerges only past a specific budget. A short probe run at 2000 epochs first surfaced a practical wrinkle: the gate moved in the correct direction (toward RBF) but too slowly, so the network still carried meaningful B-spline contamination at an epoch count where the pure-RBF pendulum recipe was already nearly converged. Giving the gate its own, faster learning rate ($15\times$ the base rate, direction untouched) closed nearly all of that gap at the same probe budget (Figure 6, left) and was carried into the full run.

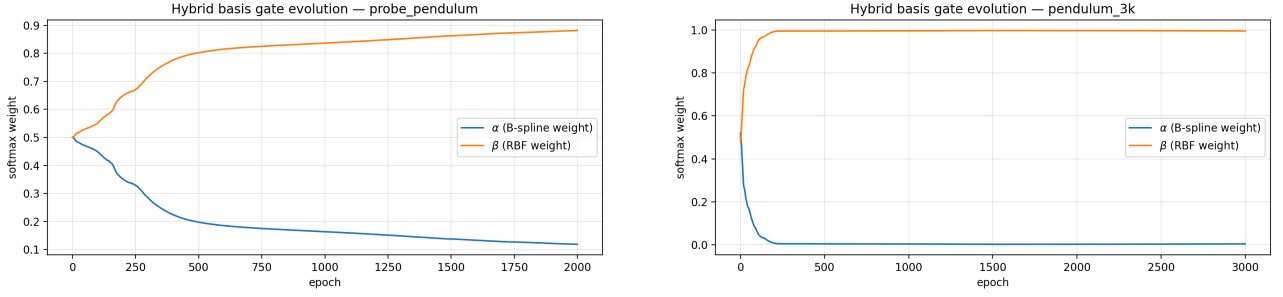


Figure 6. Left: the 2000-epoch probe run’s gate trace, showing the slow drift toward RBF that motivated the gate-learning-rate fix. Right: the gate trace for the 3000-epoch pre-overfitting-optimum run discussed below, settling to $\beta \approx 0.996$.

At the full 10,000-epoch budget, hybrid matches pure RBF’s training fit almost exactly, but its extrapolation collapses ($R^2 = -2.17$): the monitor loss bottoms out near epoch 2950 (≈ 0.054) then rises to a ≈ 0.4 plateau for the remaining 7000 epochs while training loss keeps improving – textbook overfitting, visible directly in the saved training history. The gate settles well before this onset: β reaches ≈ 0.995 by epoch 250, some 2750 epochs before overfitting begins, and pure RBF trained on the identical recipe shows no such pattern at all. Capping training at the pre-overfit optimum (epoch 3000, chosen from the 10,000-epoch run’s own point of minimum monitor loss) shows hybrid *ahead* of pure RBF at the same epoch count (0.055 vs. 0.081 full-horizon MSE) with extrapolation $R^2 = +0.580$, approaching pure RBF’s fully-converged number ($+0.647$).

The learnable hybrid basis never outperforms the better pure basis on either system at any budget tested. Its gate mechanism works exactly as designed – gradient-driven, converging differently and sensibly per system – but that alone does not buy an accuracy edge, and on the pendulum specifically a network that *starts* as a blend generalizes measurably worse over long training than one that starts pure RBF, despite an almost-identical final gate and matching training accuracy.

7 Stiffness–Solver Stability and Loss-Landscape Analysis

Numerical stiffness – a system whose fastest and slowest timescales differ by orders of magnitude – is where explicit integrators are classically expected to struggle. This study sweeps damping ratio $\mu \in \{0.1, 0.5, 1.0, 2.0, 5.0, 8.0\}$ against four explicit solvers {Euler, Midpoint, RK4, Tsit5} [7] on the damped pendulum, at a fixed step $\Delta t = 0.05$, to map which (solver, μ) combinations converge and which fail, and to test whether any failure is actually a stiffness effect.

Question	As damping ratio increases the pendulum’s dynamics stiffen – do lower-order explicit solvers genuinely fail as stiffness grows?
Expected	Higher damping ratios should push some (solver, μ) combinations outside that solver’s classical stability region, producing divergence.
Observed	Linear stability theory predicts zero divergences at every tested μ , and the sweep confirms it. Every observed failure traces to random seed, not stiffness.

7.1 The stability question resolves quickly

Linearizing at equilibrium gives eigenvalues $\lambda = (-\mu \pm \sqrt{\mu^2 - 4g/L})/2$. For every underdamped $\mu < 2\sqrt{g/L} \approx 6.26$, $|\lambda| = \sqrt{g/L} \approx 3.13$ is constant, so $h|\lambda| \approx 0.157$ at $\Delta t = 0.05$ – far inside even Euler’s stability region $h|\lambda| \leq 2$ [3]. Even the overdamped case ($\mu = 8.0$, fast eigenvalue $\lambda_{\text{fast}} \approx -7.7$) gives $h|\lambda_{\text{fast}}| \approx 0.385$, still comfortably stable. This predicts, and the sweep confirms, zero divergences across all 24 cells: not one run produced an invalid gradient at any point in training. All four solvers tested are explicit Runge-Kutta methods with a bounded stability region; no implicit solver is implemented in this project, so this analysis cannot speak to whether an implicit method would behave differently at a genuinely stiffer configuration.

7.2 What actually drives the failures: initialization

Table 9. Original 24-cell sweep (single seed 42), best training MSE and verdict.

Damping ratio μ	Euler	Midpoint	RK4	Tsit5
0.1	unstable, 1.7×10^{-2}	unstable, 3.0×10^{-2}	unstable, 2.2×10^{-2}	unstable, 2.4×10^{-2}
0.5	converged , 4.6×10^{-3}	converged , 7.9×10^{-3}	converged , 4.7×10^{-3}	converged , 9.2×10^{-3}
1.0	unstable, 1.0×10^{-1}	converged, 5.4×10^{-4}	converged, 5.3×10^{-4}	converged, 4.0×10^{-4}
2.0	unstable, 1.8×10^{-2}	unstable, 1.8×10^{-2}	unstable, 1.8×10^{-2}	unstable, 1.8×10^{-2}
5.0	converged, 6.1×10^{-5}	converged, 7.0×10^{-5}	converged, 6.9×10^{-5}	converged, 6.8×10^{-5}
8.0	converged, 2.1×10^{-5}	converged, 2.3×10^{-5}	converged, 2.6×10^{-5}	converged, 2.6×10^{-5}

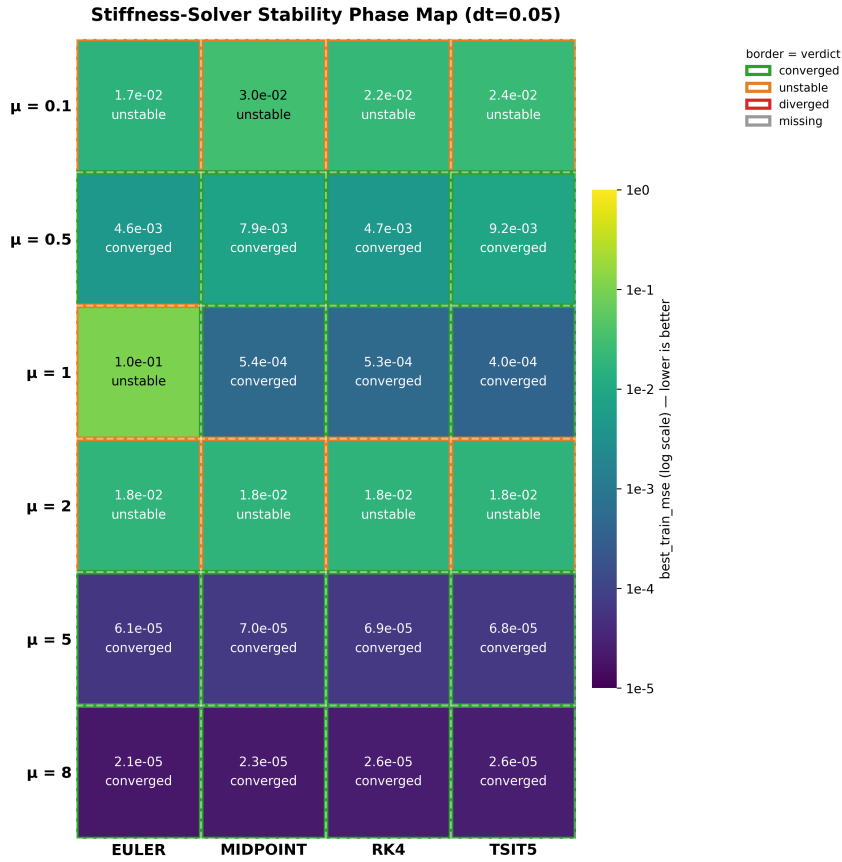


Figure 7. Stability verdict heatmap, 24-cell single-seed probe sweep, $\Delta t = 0.05$.

Two anomalies stand out: $\mu = 2.0$ fails identically across all four solvers to two significant figures, and $\mu = 1.0$ fails only for Euler. Retraining Euler at $\mu = 2.0$ with two more seeds isolates the cause (seed 42: unstable, 1.82×10^{-2} ; seed 1: unstable, 1.68×10^{-2} ; seed 7: **converged**, 7.09×10^{-5}) – the trap depends on the random initialization. Extending this to all four solvers $\times$ six $\mu \times$ three seeds (72 cells, checkpoints saved throughout) confirms the pattern generally: only the two extremes ($\mu = 0.1$, uniformly hard; $\mu \in \{5.0, 8.0\}$, uniformly easy) hold their verdict across every seed. Every transitional $\mu \in \{0.5, 1.0, 2.0\}$ has at least one of three seeds land in a bad local minimum, and at $\mu = 1.0$ two *different* seeds fail for two different reasons: seed 1 produces a universally bad initialization shared by all four solvers (MSE $\approx 9.7\text{--}9.8 \times 10^{-2}$, the same signature as the $\mu = 2.0$ shared trap), while seed 42 carries a second, Euler-specific failure on top of it. Euler’s apparent $\mu = 1.0$ weakness in Table 9 is therefore the sum of two separate effects, one shared and one solver-specific, compressed into a single misleading number.

The 24-cell sweep used one shared seed for all four solvers. All four solvers failing identically at $\mu = 2.0$ reflects four solvers inheriting the same coin flip from a shared initialization, at this step size the seed $\times \mu$ interaction determines outcome far more than solver choice does.

7.3 The mechanism: a genuine spurious equilibrium

A checkpoint saved for Euler’s trapped $\mu = 2.0$ run, evaluated at the true equilibrium and at a nearby grid-search minimum, shows the learned field f_θ has a large nonzero derivative at $(0,0)$ ($|f_\theta| = 2.74$) and a genuine, stable, spurious equilibrium 0.05–0.07 units away ($|f_\theta| = 0.0086$). This holds independently for all four solvers’ own trained models ($|f_\theta(0,0)| \in [2.65, 2.88]$ across euler, midpoint, rk4, tsit5): all four independently-trained networks converge to essentially the same wrong fixed point.

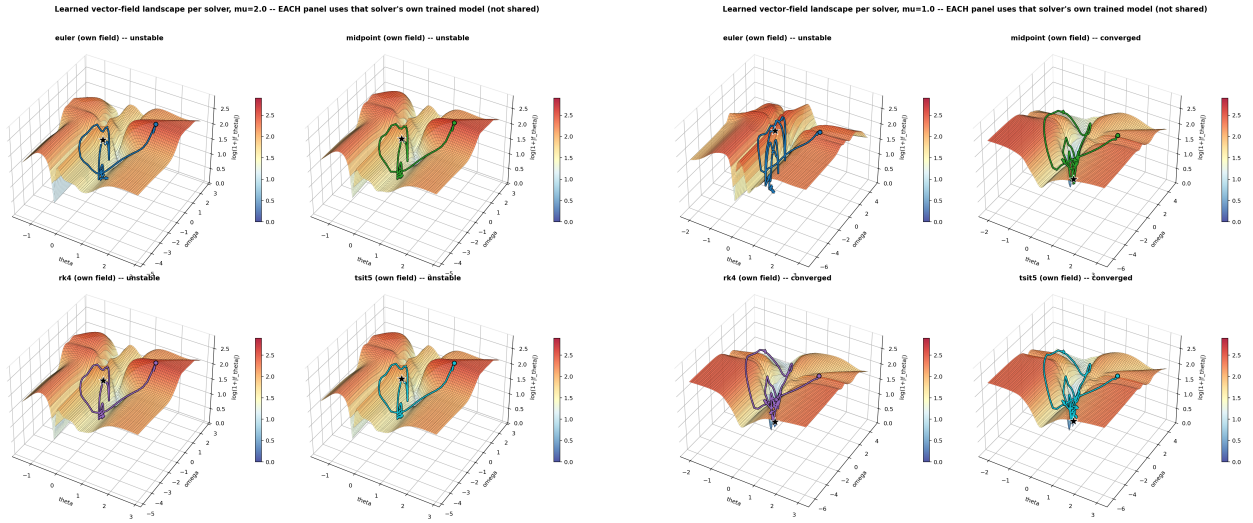


Figure 8. Learned vector-field landscape $|f_\theta(\theta, \omega)|$, per solver’s own trained model. Left ($\mu = 2.0$): every solver’s true equilibrium (star) sits partway up a slope, while a separate, lower point nearby is where each trajectory actually settles. Right ($\mu = 1.0$): Euler’s star sits high on a ridge, well apart from where its trajectory spirals down and rests; for the other three solvers, star and resting point coincide at the valley floor.

The full predicted trajectories tell the same story over time rather than in a single snapshot. Figure 9 shows every solver’s predicted angle, angular velocity, and error at $\mu = 2.0$ settling onto the same wrong, generic trajectory; Figure 10 shows the $\mu = 1.0$ case, where only Euler’s predicted angle visibly departs from the true one.

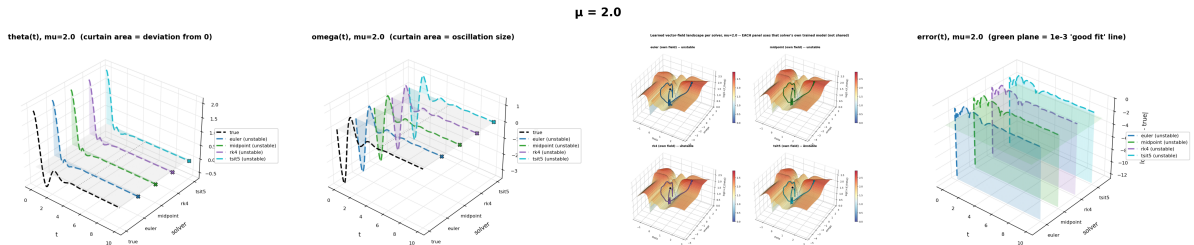


Figure 9. $\mu = 2.0$, all four solvers: predicted angle, angular velocity, and error over time, alongside each solver’s phase portrait.

7.4 Weight-space loss landscape: an independent measurement

The phase-space plots above describe the learned dynamics; a separate question is how forgiving the *training loss* is around each solution – whether a small nudge to the weights leaves the fit essentially unchanged or makes it collapse. This was measured by stepping each trained model’s weights along two random directions and re-evaluating training error on a grid, following the visualization technique of Li et al. [5] adapted to KAN-

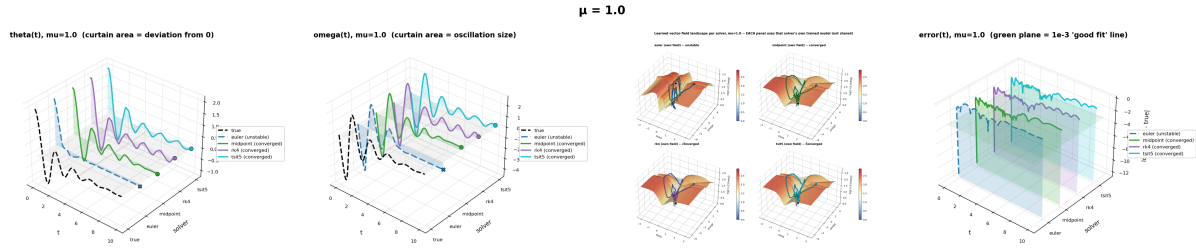


Figure 10. $\mu = 1.0$, all four solvers: the same panel layout, showing Euler’s trajectory diverging from the true angle while the other three track it closely.

ODE’s 360-parameter weight space. Every cell traces the same qualitative shape – a dip at the trained point that rises moving outward – and is summarized by its “rise”: the number of orders of magnitude the log-loss climbs by the edge of the grid.

Table 10. Loss-landscape rise, single direction-pair (seed 0).

μ	euler	midpoint	rk4	tsit5
1.0	5.53	9.03	8.99	9.12
2.0	4.23	4.24	4.22	4.20
5.0	8.75	8.75	8.76	8.76
8.0	9.14	9.15	9.12	9.12

The converged models (rise ≈ 9) sit at the bottom of a narrow, steep basin: stepping even a small distance from the trained weights in either random direction drives the log-loss up by nine orders of magnitude, so the surrounding terrain looks like a sharp valley with the solution pinned at its floor. The trapped cells (bold, rise ≈ 4 –5.5) sit in a basin roughly half as steep – the terrain around them is closer to a broad, gently sloped plain than a valley, and the same size step barely raises the loss by comparison. This is the opposite of the usual intuition that a flat minimum generalizes better: here, the flat region is where training settled for a generic, low-effort answer, and the steep, narrow basin is where it actually learned the pendulum’s dynamics.

Figure 11 renders this directly as a surface: the terrain around Euler’s trapped $\mu = 2.0$ solution is a broad, gently rolling plain, with no sharply defined bottom, in clear visual contrast to Figure 13’s converged $\mu = 1.0$ solution, which sits at the bottom of a narrow, sharply walled valley – the same contrast the rise numbers above report as a factor of roughly two, made visible. Figure 12 shows the trapped $\mu = 1.0$ case under the same solver whose $\mu = 2.0$ terrain is shown alongside it, confirming the flat terrain is a property of the trapped solution rather than of any one damping value.

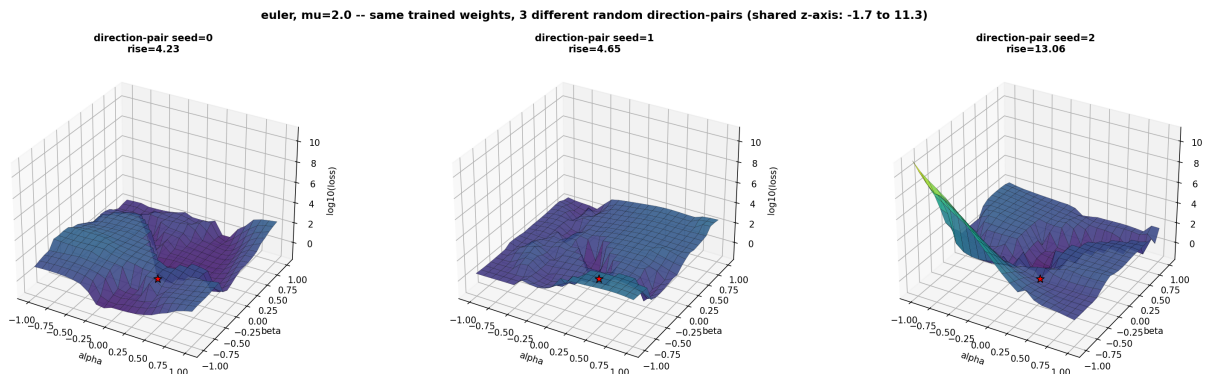


Figure 11. Euler, $\mu = 2.0$ (trapped): the training-loss surface around the trained weights, viewed under three independent random direction-pairs. The basin near the trained point (the marked star) is broad and shallow in all three views; the third panel’s steep spike on the far side is the outlier direction discussed below, not the basin itself.

Robustness check across three direction-pairs. Repeating the measurement with two more independent random direction pairs per cell (72 measurements total) found one direction-pair acting as a shared outlier

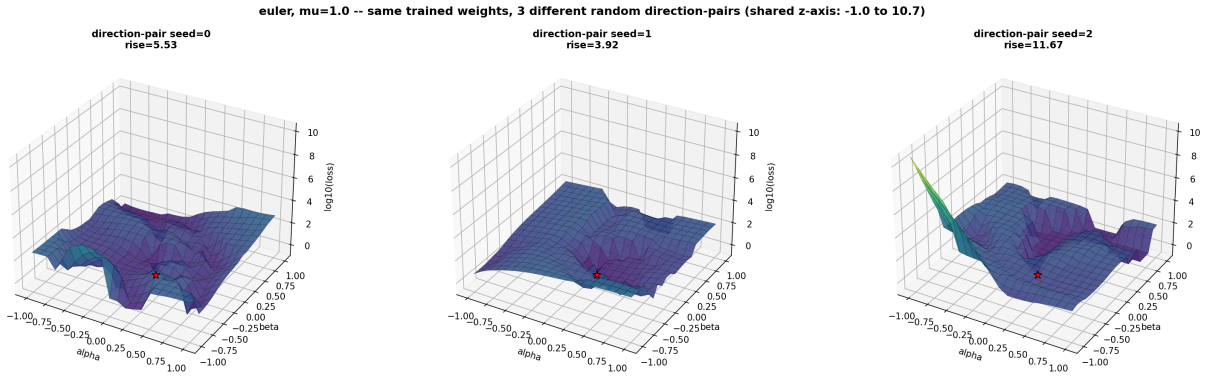


Figure 12. Euler, $\mu = 1.0$ (also trapped): the same broad, shallow basin shape recurs at a different damping value under the same solver.

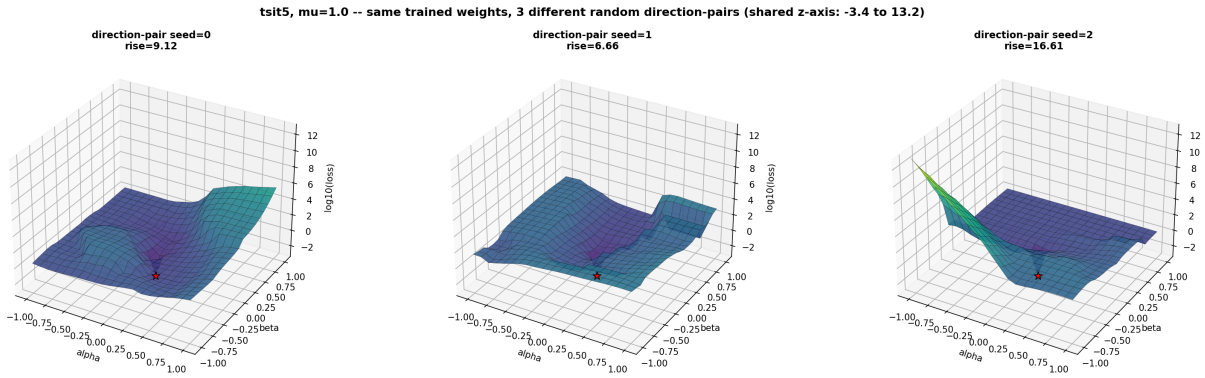


Figure 13. Tsit5, $\mu = 1.0$ (converged): the basin around the trained point is visibly narrower and drops noticeably lower, in every one of the three direction-pairs, than either trapped case above.

across nearly every cell. Because every model shares the same architecture, one particular raw random draw runs toward a steeper wall in all 24 cells at once, inflating that direction’s rise by roughly $2\text{--}3\times$ everywhere. The size of the measured effect depends on how many direction choices are averaged: the single-seed $\approx 2\times$ gap between trapped and converged cells narrows to roughly $20\text{--}30\%$ once all three direction-pairs are averaged, since the outlier direction inflates the trapped cells’ mean more than the converged cells’ mean. The ranking between cells holds regardless; only the apparent size of the gap changes. The multi-seed average is the figure to trust; the single-seed table above illustrates the shape of the effect at a scale that is easier to see.

The cells trapped by the spurious-equilibrium mechanism – $\mu = 2.0$ across all solvers, and Euler’s own $\mu = 1.0$ – sit around $20\text{--}30\%$ lower on this chart than converged cells, a modest but measured gap rather than an order-of-magnitude separation. $\mu = 0.1$ carries the “unstable” label for an unrelated reason: its trajectory never settles within the training window at all, and its bars sit in the same range as the converged $\mu \in \{5.0, 8.0\}$ cells, occasionally above them. The flatness effect belongs specifically to the spurious-equilibrium trap cases, and does not generalize to every cell the pipeline labels “unstable.”

Every divergence-free prediction from linear stability theory held exactly, so nothing in this sweep is a classical numerical-stiffness failure. The mechanism at work is a seed-dependent local minimum in a 360-parameter optimization landscape, producing a genuine, solver-independent spurious equilibrium in the learned dynamics. Three independent lines of evidence converge on this: per-solver trained-weight evaluation, phase-space vector-field visualization, and an independently-measured, direction-robust weight-space loss landscape.

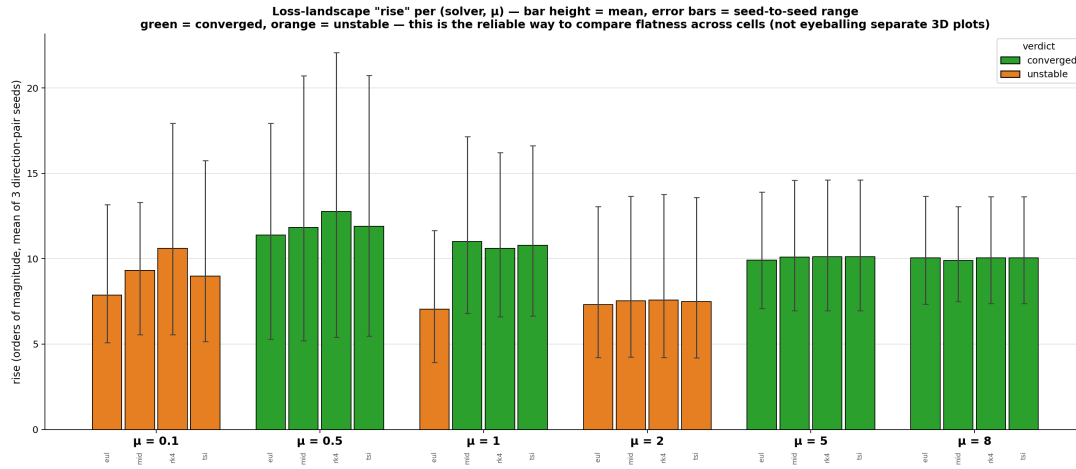


Figure 14. Mean rise per (solver, μ), 3-seed average with seed-to-seed range as error bars, colored by verdict.

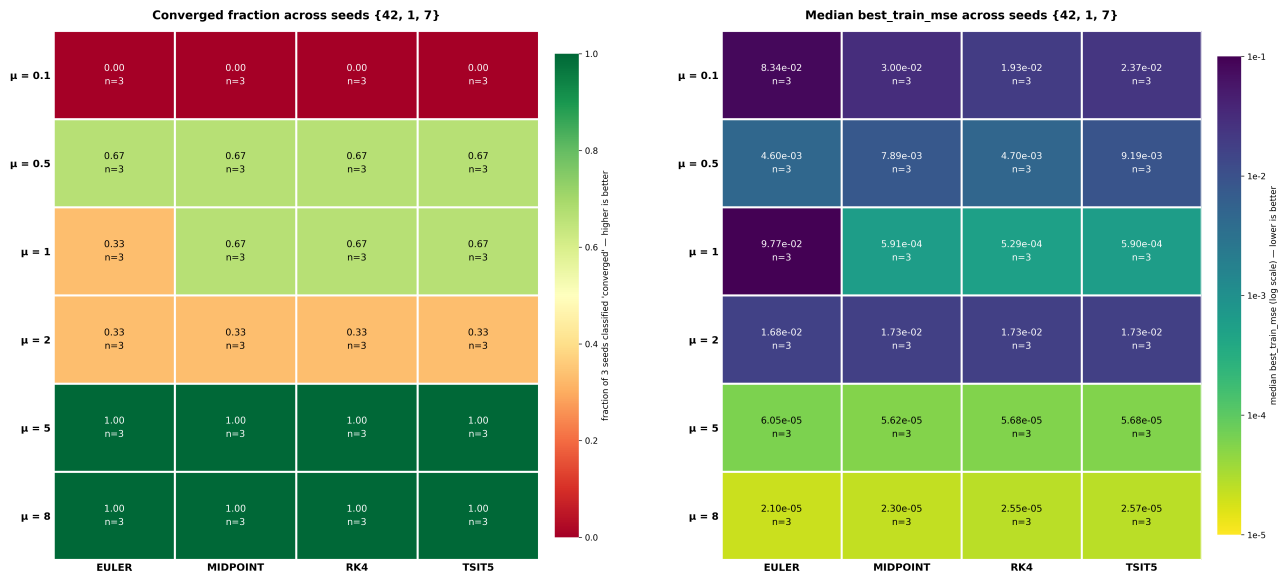


Figure 15. Converged fraction (left) and median training MSE (right) over 3 seeds, all 4 solvers $\times$ 6 μ (72 cells). The two agree everywhere: low converged-fraction cells are exactly the cells with elevated median error.

8 Sparse Regression Comparison and Real-World Epidemic Validation

This section compares two fundamentally different ways of discovering governing equations from data – sparse symbolic regression and gradient-based neural fitting – and separately tests whether the cross-domain stability fixes of Section 3 generalize beyond the mechanistic system that produced them.

8.1 Noise robustness: KAN-ODE vs. sparse symbolic regression

Sparse Identification of Nonlinear Dynamics (SINDy) [1] fits a governing equation as a sparse linear combination of candidate terms drawn from a fixed library – here, every polynomial in x and y up to degree 2, i.e. $\{1, x, y, x^2, xy, y^2\}$. Coefficients are estimated by sequentially thresholded least squares: an ordinary least-squares fit is performed against numerically estimated derivatives, every coefficient below a fixed threshold is zeroed out, and the fit is repeated on the surviving terms until the active set stops changing. This is what makes the recovered model sparse and, in principle, human-readable – unlike the KAN’s 240 free parameters, SINDy returns a short list of surviving terms with numeric coefficients attached, directly comparable to the true equation’s own coefficients. The coefficient threshold is held fixed across the entire noise sweep so that the comparison measures noise sensitivity rather than per-level tuning. It is fit against the same noise-corrupted Lotka-Volterra observations the earlier KAN checkpoints (Section 3) were trained on, scored against clean ground truth.

Question	How does a sparse, symbolic method compare to a dense, 240-parameter neural method as observation noise increases?
Expected	Sparse regression’s tiny library should make it more robust to noise; KAN-ODE’s larger parameter count should make it more accurate.
Observed	Both expectations hold, but the KAN-ODE advantage collapses $170\times$ across the sweep, from $1110\times$ at zero noise to $6.5\times$ at $\sigma = 0.1$.

Table 11. Extrapolation MSE vs. observation noise σ .

Noise level σ	Sparse regression	KAN-ODE	KAN-ODE advantage
0.00	9.90×10^{-2}	8.92×10^{-5}	$1110\times$
0.01	1.58×10^{-1}	1.22×10^{-3}	$130\times$
0.05	1.48×10^{-1}	2.54×10^{-2}	$5.8\times$
0.10	8.89×10^{-1}	1.37×10^{-1}	$6.5\times$

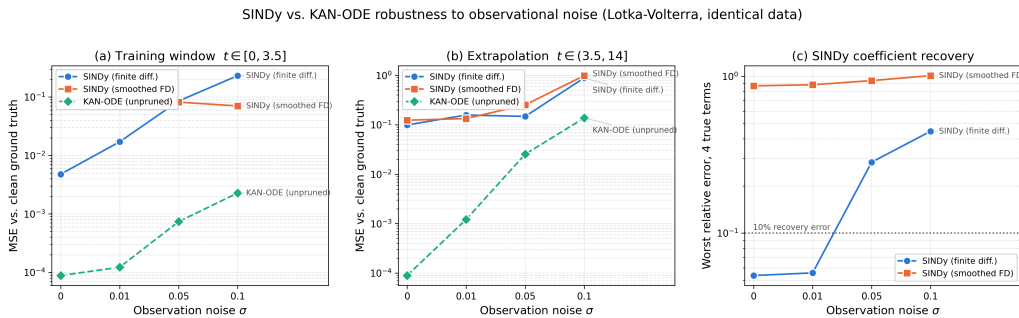


Figure 16. Training and extrapolation MSE vs. noise level, both methods.

KAN-ODE wins on absolute accuracy at every level, but the margin collapses $170\times$ across the sweep (sparse regression degrades $9.0\times$ from $\sigma = 0$ to 0.1 ; KAN-ODE degrades $1540\times$). The sparse regression’s six-term library caps how much noise can move it; the KAN’s 240 free parameters fit the noise directly and the error compounds through the solver. Per-term recovery shows the bilinear cross-term (xy) is recovered to within 5.7% at every noise level – exactly the term a single-layer KAN edge, being strictly univariate, cannot represent directly, since each edge is a function of only one input variable and a layer sums its edges additively. This asymmetry – sparse regression writes xy as a literal library column, while a KAN can only approximate it through composition across a hidden layer – means a term-by-term symbolic comparison would be comparing a

coefficient against a distributed representation, so the comparison used here is trajectory reconstruction instead.

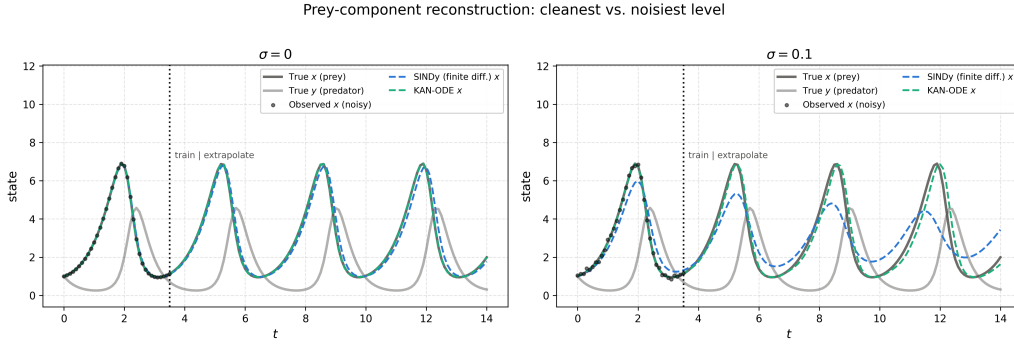


Figure 17. Reconstructed trajectories at $\sigma = 0$ and $\sigma = 0.1$: sparse regression loses amplitude under noise; KAN-ODE keeps amplitude but drifts in phase.

8.2 Edge pruning: testing whether a term-by-term comparison is even possible

Part of the original motivation for a smooth, globally-supported basis such as RBF is that a sparse, pruned KAN edge structure could in principle be read back as a closed-form symbolic law, the same interpretability goal sparse regression achieves by construction. A KAN-ODE analogue of a sparse coefficient list would come from pruning weak edges and reading off what survives. Ranking each edge by mean spline-coefficient magnitude and masking the weakest, applied to the existing (never retrained) noise-sweep checkpoints, removes the two weakest of forty edges and raises the noise-free full-horizon MSE from 8.90×10^{-5} to 3.43 – a factor of 3.9×10^4 , worse than predicting the constant dataset mean (2.91). Every checkpoint was trained with no sparsity-inducing regularization applied at all, so nothing during training ever asked the network to concentrate its fit onto a few edges; the flat plateau at ≈ 3.5 – 3.8 across all pruning levels of 25% or more is the signature of a model where every edge carries part of the fit, confirming that the pruning procedure itself works correctly on a network that has no prunable structure to find.

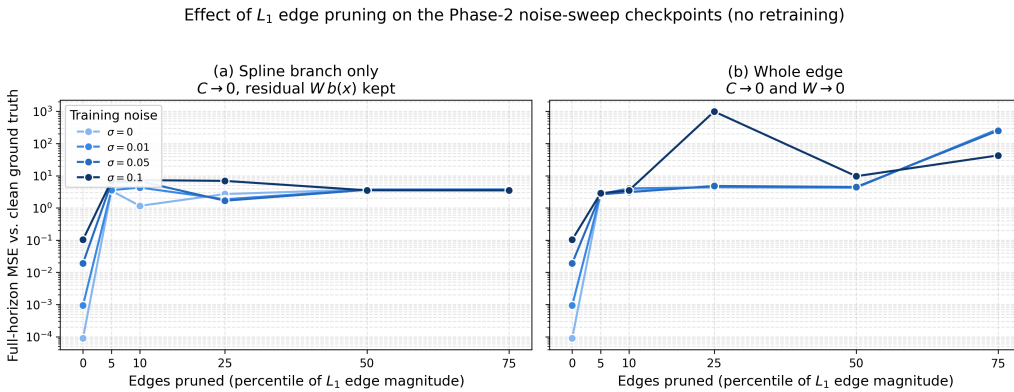


Figure 18. Full-horizon MSE vs. edge-pruning fraction, all four noise levels.

8.3 Testing whether SIR's fixes transfer to a non-mechanistic epidemic curve

A real, asymmetric log-normal outbreak curve, never previously used and generated by no compartmental ODE, tests whether the SIR fixes of Section 3 generalize by mechanism rather than by coincidence of the SIR system's own structure. The same init-time diagnostic used to root-cause SIR reproduces an almost identical speed-mismatch signature ($18.77\times$ vs. SIR's $19\times$), justifying a time rescaling by a factor of 24 *a priori*, before any training.

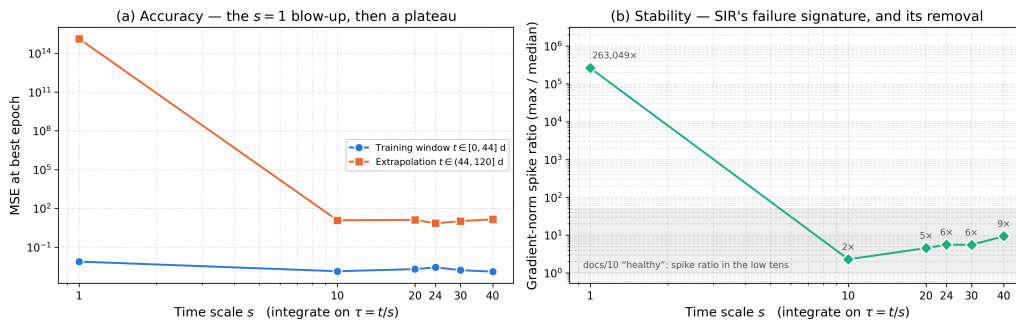
Question	SIR's three stability fixes were each derived from a property of the SIR generator itself – do they still work on data no compartmental model produced?
Expected	Only the unit-rescaling fix, which assumes nothing about the data's structure, should transfer; the two structural fixes should not.
Observed	One in three predictions was wrong: the vanishing-field gate transfers and is the best single option, despite being predicted inapplicable.

Table 12. Which of SIR's three fixes transfer to a non-mechanistic curve.

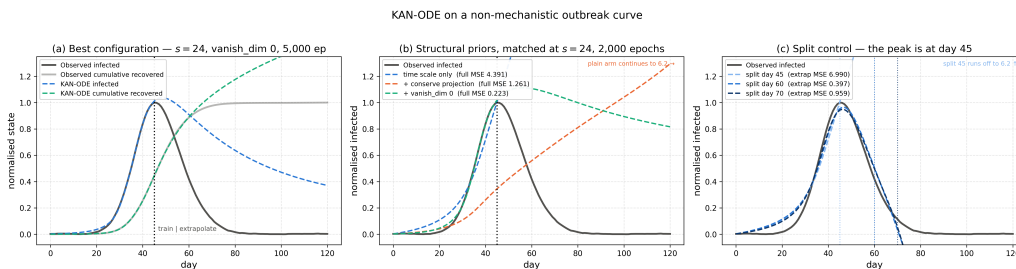
Fix	Precondition	Holds?	Outcome
Time rescaling	None – a unit change	Yes	Removes a $263,049\times$ gradient spike
Conservation via sub-space projection	An exact invariant, $\sum y = c$	No ($\sum y$ spans 0.002–1.54)	Inapplicable; $25\times$ worse fit
Vanishing-field gate at the equilibrium point	$y_d = 0$ is a manifold of equilibria	Yes (unexpectedly)	Best single option, $47\times$ better fit

The vanishing-field gate was *predicted* inapplicable: the infected fraction at $t = 0$ is 0.0027, seemingly too small to avoid freezing the field entirely. A control arm overturns that prediction: gating the learned field by the initial state correctly encodes that the rate of change is proportional to current prevalence, and is the only configuration that reproduces the existence of a peak (day 48 predicted vs. true day 45) from a training window that never observed one.

Time-scale sweep on the empirical outbreak (2,000-epoch probes)

**Figure 19.** Time-rescaling sweep: accuracy and gradient-spike ratio vs. the rescaling factor.

The extrapolation runaway traces to the train/test split. The default split lands exactly on the infected peak (0% of the decay phase is in the training window), so every model is asked to predict a monotone decay it has never observed. Moving the split just 15 days later collapses the predicted maximum from $6.18\times$ the true peak to 0.964 (true maximum = 1.0), turning over at day 47 against a true day 45 – the runaway is a property of where the window ends, not of the method fitting the data.

**Figure 20.** Real epidemic fit: default split (runaway), the vanishing-field-gate arm, and the fifteen-day-later split control.

Time rescaling transfers unconditionally because it is a unit change with no structural precondition. The other two fixes are structural priors that transfer exactly insofar as their structure is actually present in the new data: the conservation projection’s precondition is confirmable in advance by directly measuring the compartment sum, while the vanishing-field gate’s precondition turned out to require running the control arm to settle – argument alone predicted the wrong outcome. That arm cost 27 minutes and overturned the prediction.

9 Validation Status: Epoch-Budget Sensitivity and Implementation Audit

Two supervisor-requested validation tasks were scoped as follow-on work to the benchmark ablation (Section 3): an epoch-budget sensitivity check, and a baseline-vs-paper implementation audit. Both are fully designed and their execution infrastructure is complete, but neither has been run to completion as of this report.

9.1 Epoch-budget sensitivity check

The benchmark ablation trained all 24 Lotka-Volterra configurations for a fixed 10,000 epochs; whether that budget is long enough for the reported *rankings* to be trustworthy (as opposed to absolute numbers) was never independently checked. Five representative configurations were selected to re-train at $5 \times$ budget (50,000 epochs): Euler (the one solver that does not reach 10^{-4} within 10,000 epochs), Tsit5 with the RBF basis (the project’s reference point), B-spline (the near-tie with RBF in Table 2), and both MLP baselines (the parameter-matched version, and the version matching the paper’s literal specification, which fails to train at all under this project’s protocol). Ready-to-run training scripts for all five configurations already exist, targeting the same cloud CPU infrastructure used elsewhere in this project, at a measured ≈ 17.0 h total CPU-time (≈ 9.3 h wall-clock run in parallel). As of this report, none of the five have been executed against that budget; the design is complete and awaits compute time.

9.2 Baseline-vs-paper implementation audit

A field-by-field comparison of this project’s recorded training recipe (a two-layer Kolmogorov-Arnold Network with 10 hidden nodes, a 5-point RBF grid, the Tsit5 solver, step size $\Delta t = 0.1$, learning rate 2×10^{-3} , and 10,000 training epochs) against the original paper’s own stated methodology [4] is a reading and documentation task that requires no additional compute, and has not yet been completed. One piece of evidence is already in hand: the paper’s literal MLP baseline ($[2, 50, 2]$, tanh) fails to train under this project’s protocol at all (Section 3), which points at an architecture/activation mismatch independent of epoch budget for that specific baseline – the planned 50,000-epoch run of that exact configuration (above) is what would convert this from suggestive evidence into a confirmed answer.

Both validation tasks are fully designed, justified, and infrastructure-ready. Execution remains outstanding. The five studies’ findings (Sections 4–8) are stated in this report as measured under a fixed 10,000-epoch budget and this project’s own recorded recipe, leaving the epoch-budget and paper-implementation-parity questions as open follow-up work.

10 Discussion

Three patterns recur across studies that were designed independently and address unrelated questions. Their recurrence is itself informative about how this class of model should be studied going forward.

A reasoned hypothesis still requires a control arm. Three separate studies advanced a specific, mechanistically plausible hypothesis and then tested it directly: gradient-norm roughness as the cause of Euler’s convergence floor (Section 5), SiLU’s representational limit as the cause of the pendulum’s early failure (Section 3), and the predicted inapplicability of the vanishing-field gate to non-mechanistic data (Section 8). Each hypothesis had a clear argument behind it before any run was launched, and each was overturned by the run

itself. The result reported in every one of these three cases is the outcome of the control arm, and the original argument survives only as the motivation for running it.

A single seed and a single train/test split account for results that first looked like properties of the solver or the method. The stiffness-map failures at $\mu \in \{0.5, 1.0, 2.0\}$ (Section 7) and the epidemic-fit extrapolation runaway (Section 8) both appeared, on first inspection, to indict a solver or a modeling approach. Retraining with additional seeds, in the stiffness-map case, and shifting the train/test split by fifteen days, in the epidemic case, resolved both into an artifact of one specific run’s random initialization or one specific window boundary. Neither observation survives as a claim about solver order or about KAN-ODE’s capacity once the seed or the split is varied.

Measuring a quantity before training is more informative than tuning after a failed run. The SIR divergence (Section 3), the real-epidemic-data fit (Section 8), and the stiffness map’s spurious equilibrium (Section 7) were each root-caused by evaluating the model’s own initial field, or its trained field at a candidate equilibrium, and comparing that number against the system’s true scale. In every case this produced a specific, falsifiable prediction – a rescaling factor, a fix that should or should not transfer, an equilibrium location – that a subsequent short run confirmed or refuted directly, in contrast to the open-ended search a hyperparameter sweep would have required.

11 Conclusion

This project reproduced a published KAN-ODE recipe, diagnosed and fixed two genuine cross-domain training failures, and ran five further studies that each isolated a specific question about solver behavior, gradient dynamics, basis-function design, and stiffness. Every quantitative claim in this report traces to a checked-in metrics file or a regenerable figure, and every study that tested a specific mechanistic hypothesis reports the outcome of a direct control arm rather than the hypothesis alone.

The validation stage (Section 9) is the project’s principal open item: the results in Sections 4–8 rest on a single random seed and a fixed 10,000-epoch training budget, and the epoch-budget sensitivity check and paper-implementation audit designed to test those two assumptions have not yet been executed. Completing them is the natural next step for extending the confidence of every result reported here.

References

- [1] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016.
- [2] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- [3] Germund Dahlquist. A special stability problem for linear multistep methods. *BIT Numerical Mathematics*, 3(1):27–43, 1963.
- [4] Benjamin C. Koenig, Suyong Kim, and Sili Deng. KAN-ODEs: Kolmogorov-arnold network ordinary differential equations for learning dynamical systems and hidden physics. *Computer Methods in Applied Mechanics and Engineering*, 432:117397, 2024. arXiv:2407.04192.
- [5] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. Visualizing the loss landscape of neural nets. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- [6] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. KAN: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.
- [7] Ch. Tsitouras. Runge–kutta pairs of order 5(4) satisfying only the first column simplifying assumption. *Computers & Mathematics with Applications*, 62(2):770–775, 2011.